SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA5116 - Cryptography and Network Security

ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.
CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-

exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from (g^a)^b ≡ (g^b)^a (mod p).
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.
Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

CSA5116 – Cryptography and Network Security

ASSIGNMENT 2 (CO2)

Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network

Conventions and reproducibility. All cores are my own Python code (crypto2.py): DES, 3DES, Blowfish, RC5, five block-cipher modes, GCD/extended Euclid/modular exponentiation, trial division/Fermat/Miller–Rabin, RSA with OAEP and signatures, Diffie–Hellman, elliptic curves with ECDH/ECDSA and a BB84 simulation. Each cipher was checked against published test vectors (Q1). The standard library is used only for SHA-256 and seeded random numbers. Experiments are in exp2.py and the figures in figs2.py; every number below comes from them, with fixed seeds. All keys, moduli and messages are toy or student-made test data; no real patient data was used and none of this code should protect real data.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 marks)

Cipher	Structure and block	Key	Rounds / round operation	Characteristics and legacy limits
DES	Feistel network, 64-bit block	56 effective bits (64 with parity)	16: expand R (32→48), XOR subkey, 8 S-boxes (6→4 bits), permutation P, swap halves	Key space 2^{56}; average search 2^{55} keys; built for hardware, slow in software; broken by exhaustive search (a purpose-built machine did it in about two days in 1998).
3DES (EDE)	DES three times: $E_{K3}(D_{K2}(E_{K1}(P)))$, 64-bit block	112 or 168 nominal bits; about 112 effective (meet-in-the-middle)	48 DES rounds per block	Three times the cost of DES; the 64-bit block allows birthday collisions; retired by NIST for new use after 2023.
Blowfish	Feistel, 64-bit block	32–448 bits (128 used)	16: L ⊕= P_i; R ⊕= F(L), F = ((S_0[a] + S_1[b]) ⊕ S_2[c]) + S_3[d] mod 2^{32}	Key-dependent S-boxes (4 KB) and P-array built by encrypting 521 blocks: slow key setup, fast bulk speed; 64-bit block limits data per key.
RC5-32/12/16	Word-oriented Feistel-like, 64-bit block (w = 32)	up to 2040 bits (128 used)	12: A = ((A ⊕ B) <<< B) + S[2i]; B = ((B ⊕ A) <<< A) + S[2i+1]	Data-dependent rotations; table of only 26 words (104 bytes); fast key setup; less analysed, 12 rounds is a minimum; 64-bit block.

Implementation and verification. DES, 3DES, Blowfish (constants generated from the hexadecimal digits of π by my own program) and RC5 were all implemented and decrypt correctly. Outputs match the published vectors:

Cipher	Key and plaintext	Published ciphertext	My program
DES	key 133457799BBCDFF1, P 0123456789ABCDEF	85E813540F0AB405	85E813540F0AB405
Blowfish	key 00..00, P 00..00	4EF997456198DD78	4EF997456198DD78
Blowfish	key FF..FF, P FF..FF	51866FD5B85ECB8A	51866FD5B85ECB8A
RC5-32/12/16	key 00*16, P 00*8	21A5DBEE154B8F6D	21A5DBEE154B8F6D
RC5-32/12/16	key 915F..CE91, P 21A5DBEE154B8F6D	F7C013AC5B2B8952	F7C013AC5B2B8952
3DES (EDE)	3 keys, P 'PT20481 '	round trip	486CB716B3AC073F -> PT20481

Block versus stream. A block cipher maps a fixed block (64 bits here) under a key and needs a mode to handle long data; a stream cipher (or a block cipher in OFB/CTR mode) XORs a key-derived keystream with the data byte by byte. *Appropriate use:* a block cipher in CBC or CTR mode for stored records and file transfers; a stream construction for live telemetry from medical devices (no padding, no latency, one-bit errors), with the rule that a keystream must never be reused.

Mode	IV / nonce requirement	Pattern leakage	Error propagation (1 flipped ciphertext bit; measured, 16 blocks)	Parallelism
ECB	none	identical blocks give identical ciphertext	block 5 only (33 bits wrong)	encrypt and decrypt
CBC	unpredictable IV, never reused with a key	hidden if IV is fresh; same IV reveals equal first blocks	block 5 garbled + 1 bit in block 6 (35 bits)	decrypt only
CFB	unique IV	hidden	1 bit in block 5 + block 6 garbled (34 bits)	decrypt only
OFB	unique IV (reuse repeats the keystream)	hidden	1 bit only (1)	none (keystream sequential)
CTR	unique nonce per key (counter must never repeat)	hidden	1 bit only (1)	encrypt and decrypt

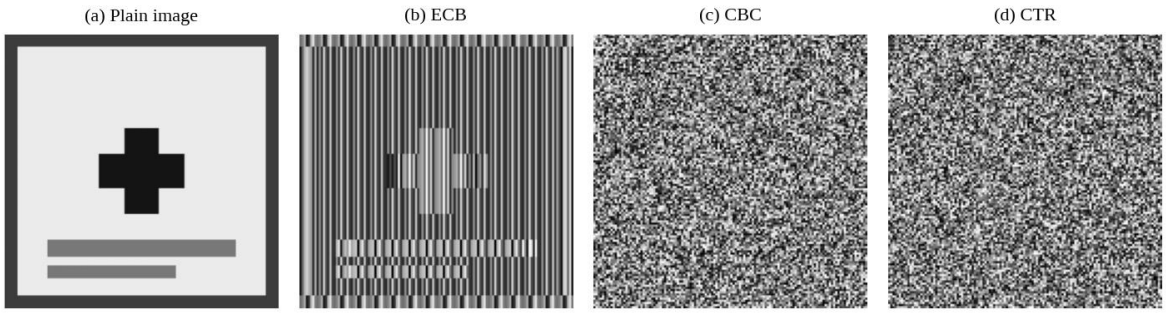


Figure 1. A 128×128 test image (a) encrypted with Blowfish in ECB (b), CBC (c) and CTR (d) mode.

Pattern and IV/nonce demonstration. The image has large flat areas, so 2038 of its 2048 8-byte blocks are repeats. ECB keeps all 2038 repeats and the outline of the cross and bars stays visible (Fig. 1b); CBC and CTR give 0 repeated blocks and look like noise. *Nonce reuse:* two messages encrypted with the same CTR key and nonce satisfy $C_1 \oplus C_2 = P_1 \oplus P_2$ (verified). Knowing only the 29-byte start of the prescription ("PRESCRIPTION: AMOXICILLIN 500") gave the start of the other message, "DIAGNOSIS: SEVERE PENICILLIN"; if one message is fully known the whole other message is recovered (60 of 60 bytes). In CBC, reusing an IV makes the equal first blocks of two messages visible (measured: equal = true).

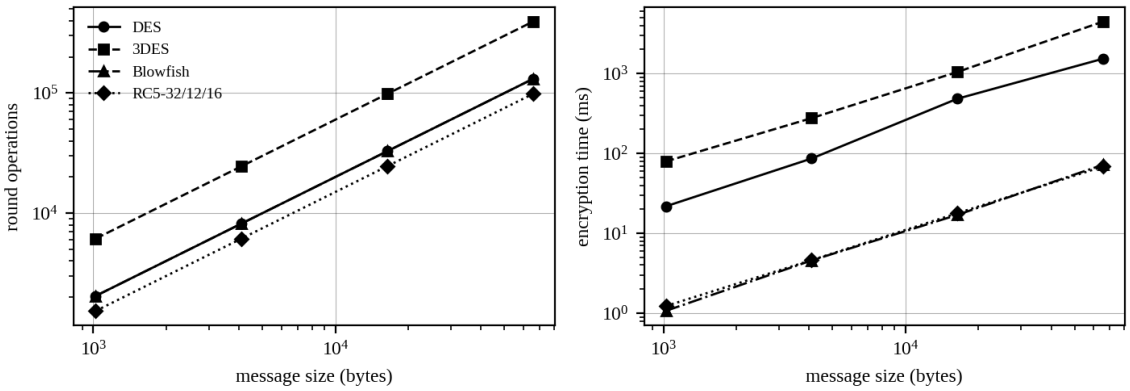


Figure 2. Round operations (left) and measured time (right) against message size for the four ciphers (my Python code).

Cipher	Rounds per block	Round operations for 64 KB	1 KB ms	4 KB ms	16 KB ms	64 KB ms	Key setup ms
DES	16	131,072	21.8	86.43	483.91	1534.38	0.182
3DES	48	393,216	79.48	276.85	1046.32	4471.42	0.538
Blowfish	16	131,072	1.07	4.58	17.01	72.25	8.326
RC5-32/12/16	12	98,304	1.23	4.62	17.86	68.59	0.09

Reading the numbers. Time and operation counts grow linearly with size (slope 1 on the log–log graph); only the constant differs. Software DES is slow (1534.38 ms for 64 KB) because bit permutations are cheap in hardware but costly in software; 3DES is about three times slower (4471.42 ms) and Blowfish and RC5 are about twenty times faster than DES (72.25 and 68.59 ms). Blowfish pays for its speed with a 8.326 ms key setup against 0.09 ms for RC5. A reduced DES search (12 unknown key bits) found the key after 3825 trials in 1.01 s (about 3,804 keys/s in Python); at that rate 2^{55} keys would take about 301,000 years in Python, which only shows that software is the wrong tool: dedicated hardware reduced it to days.

Ranking for this network and legacy limitations. (1) **RC5** (use at least 16 rounds): 104-byte table, instant key setup, and speed close to Blowfish suit constrained clinic devices that re-key for every file. (2) **Blowfish**: fastest bulk speed and no practical attack on the full cipher, but 4 KB of tables and slow key setup; best on the hospital server. (3) **3DES**: still 112-bit strong but three times slower and officially retired. (4) **DES**: broken by exhaustive search. *A larger key space is not proof of security:* Blowfish accepts a 448-bit key, yet its 64-bit block means about $N^2/2^{65}$ chance of a repeated block after N blocks (0.05% after 1 GiB, 3.1% after 8 GiB, 50% after 32 GiB), so a key must never encrypt more than about 1 GiB, and ECB/CBC rules on IVs still apply. *Recommendation:* use CTR or CBC (never ECB) with a fresh key per file and a unique nonce; use a stream construction for live telemetry; and where hardware allows, replace all four by AES-256 in an authenticated mode (GCM), which has a 128-bit block and gives integrity.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 marks)

Routine	Pseudocode (my implementation)
GCD / extended Euclid	EGCD(a, b): (x0, y0, x1, y1) ← (1, 0, 0, 1) while b ≠ 0: q ← a div b; (a, b) ← (b, a − q·b); (x0, x1) ← (x1, x0 − q·x1); (y0, y1) ← (y1, y0 − q·y1); steps ← steps + 1 return (a, x0, y0): a·x0 + b·y0 = gcd(a, b)
Modular inverse	INV(a, m): (g, x, y) ← EGCD(a mod m, m); if g ≠ 1 → error "no inverse"; return x mod m trial version: for x = 1 … m−1: if (a·x) mod m = 1 return x
Square-and-multiply	POW(b, e, m): r ← b; for each bit of e after the leading 1: r ← r·r mod m; if bit = 1: r ← r·b mod m; return r naive version: r ← 1; repeat e times: r ← r·b mod m
Miller–Rabin	write n − 1 = 2^s·d (d odd); for each base a: x ← a^d mod n; if x ∈ {1, n−1} continue; repeat s−1 times: x ← x^2 mod n, if x = n−1 break; otherwise n is composite

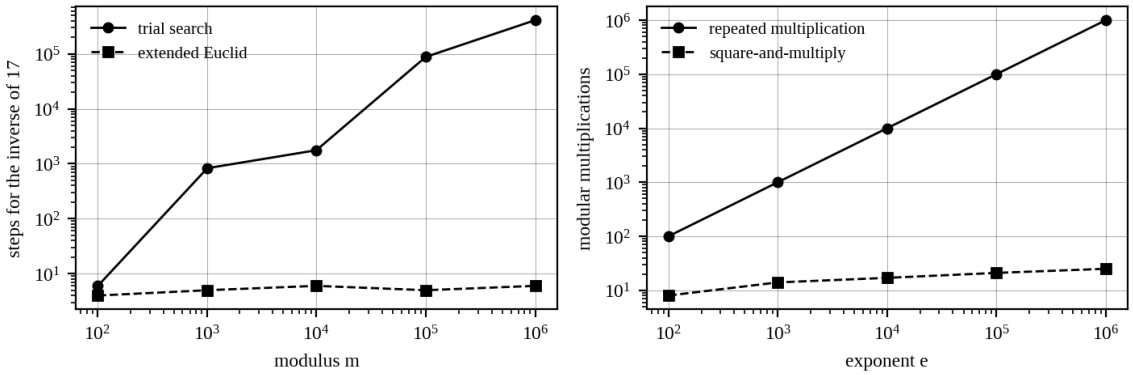


Figure 3. Inverse of 17: trial search against extended Euclid (left); repeated multiplication against square-and-multiply (right).

Comparisons. Inverse of 17 modulo m: trial search needs 411,766 steps at m = 1,000,003 while extended Euclid needs 6 (about log m); times 30392.1 µs against 5.89 µs. Computing 7^e mod (10^9+7): repeated multiplication needs e − 1 multiplications (1,000,000 for e = 10^6) against 25 for square-and-multiply (≈ 1.5·$\log_2$ e); time 71.934 ms against 0.0113 ms. Results always matched Python's pow().

Euler's totient and the condition on e. φ(n) counts the integers in 1…n coprime to n; for n = pq, φ(n) = (p−1)(q−1). Euler's theorem, $a^{\varphi(n)} \equiv 1 \pmod n$ for gcd(a, n) = 1, makes decryption work: $M^{ed} \equiv M^{1+k\varphi} \equiv M$. The exponent $d = e^{-1} \bmod \varphi(n)$ exists only if gcd(e, φ(n)) = 1. For φ = 60·52 = 3120 = 2^4·3·5·13 my gcd test gave gcd(e, 3120) = 3, 5, 13 and 15 for e = 3, 5, 13 and 15 (invalid) and 1 for e = 7, 17 and 65537 (valid).

n	Truth	Trial division steps	Fermat base 2	Fermat mults	Coprime bases passing Fermat	Miller–Rabin 2,3,5,7	M–R mults
561	composite	2	passes	11	100%	composite	10
1,105	composite	3	passes	12	100%	composite	11
1,729	composite	4	passes	13	100%	composite	12
2,465	composite	3	passes	14	100%	composite	13
2,821	composite	4	passes	14	100%	composite	13
6,601	composite	4	passes	17	100%	composite	16
7,919	prime	44	passes	21	100%	prime	80
104,729	prime	162	passes	22	100%	prime	80
1,000,003	prime	500	passes	26	100%	prime	100
1,000,001	composite	51	rejects	25	0%	composite	24

Primality-test evidence. The Carmichael numbers 561, 1105, 1729, 2465, 2821 and 6601 pass the Fermat test for base 2 and for 100% of the bases coprime to them, so Fermat cannot reject them with such bases; Miller–Rabin rejects all of them. Exhaustive count over the 40,408 odd composites below 100,000: Fermat base 2 accepted **78**, Fermat bases 2, 3, 5, 7 accepted 3, Miller–Rabin base 2 accepted 16 (strong pseudoprimes), Miller–Rabin bases 2, 3, 5, 7 accepted **0**; there are 16 Carmichael numbers below 100,000. Trial division is always correct but costs about √n/2 steps: 505, 28,588 and 481,766 steps for primes of 20, 32 and 40 bits, against 160, 220 and 280 multiplications for five Miller–Rabin rounds (linear in the bit length).

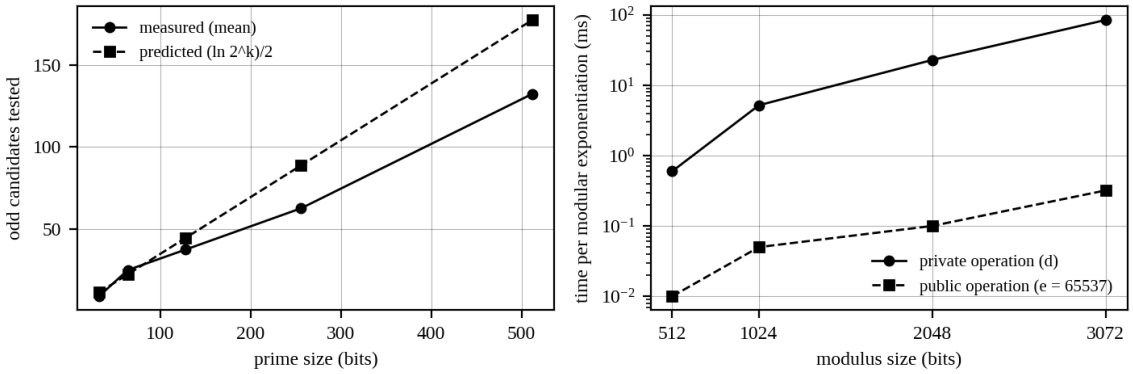


Figure 4. Odd candidates tested per prime, measured against ln(2^k)/2 (left); time per modular exponentiation against modulus size (right).

Prime density. By the prime number theorem the chance that a random number near x is prime is about 1/ln x, so a random odd k-bit candidate is prime with probability ≈ 2/(k·ln 2) and about (k·ln 2)/2 odd candidates are tested on average. Measured density in $[2^k, 2^{k+1})$: k = 10: 0.1338 (predicted 0.1363); k = 14: 0.0984 (predicted 0.0989); k = 18: 0.0778 (predicted 0.0776); k = 22: 0.0639 (predicted 0.0639). Measured candidates per prime for 32, 64, 128, 256, 512 bits: 8.8, 24.8, 37.3, 62.5, 132.3 against predicted 11.1, 22.2, 44.4, 88.7, 177.4; a small-prime sieve removes about 85% of candidates before any Miller–Rabin round.

RSA modulus	Candidates tested	M–R calls	Modular multiplications	Key generation s	Private op ms	Public op ms	Trial-division steps
512 bits	56	11	7,202	0.01	0.6	0.01	2^{256}
1024 bits	680	120	97,978	0.14	5.2	0.05	2^{512}
2048 bits	943	144	233,040	0.87	22.9	0.1	2^{1024}
3072 bits	332	54	142,514	0.98	84.8	0.32	2^{1536}

Engineering decision. Moving from classroom values to 1024, 2048 and 3072 bits, trial division and naive exponentiation become impossible (2^{512} to 2^{1536} steps), whereas Miller–Rabin and square-and-multiply need only a few thousand multiplications of growing size. The private operation cost rose from 5.2 to 22.9 to 84.8 ms (the theory predicts about ×8 per doubling of the size; Python's faster big-number multiplication gives less), and the number of candidates varies a lot from key to key (for the 3072-bit key only 332 against an expected ≈ 532). Decision: use my own Extended Euclid, square-and-multiply and Miller–Rabin with a small-prime sieve and random bases from a secure random generator; choose e = 65537 (17 multiplications for the public operation); use moduli of at least 2048 bits today and 3072 bits for data that must stay secret beyond 2030; and add weak-key tests (Q7).

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 marks)

Hand-checkable example. p = 61, q = 53, n = 3233, φ = 3120, e = 17, d = 17^{-1} mod 3120 = 2753 (17·2753 = 46,801 = 15·3120 + 1). For M = 65: $65^2 \equiv 992$, $65^4 \equiv 1232$, $65^8 \equiv 1547$, $65^{16} \equiv 789$ (mod 3233), so C = 65^{17} = 789·65 mod 3233 = **2790**; decryption 2790^{2753} mod 3233 = **65**. The program agrees (5 multiplications for encryption). RSA keys of 1,024–3,072 bits (Q2) use the same code.

Predict, test, compare. The prediction column was written before running the tests.

Test	Prediction	Observed result	Match
Encrypt M = 65 twice	same ciphertext both times (deterministic)	2790 and 2790	yes
Boundary M = 0, 1, n-1	C = 0, 1, n-1 (fixed points)	0, 1, 3232	yes
M = n + 5 (outside range)	decrypts to 5, not n + 5	5	yes
e = 15 with phi = 3120	no inverse, key rejected	no inverse: gcd(15,3120)=15	yes
p = q = 61 (n = 3721, phi taken as (p-1)(q-1))	many messages decrypt wrongly	61 of 3721 correct (1.6%)	yes
p = 91 (composite 7 x 13) used as prime, q = 53	many messages decrypt wrongly	4823 of 4823 correct (100.0%)	NO – prediction wrong
e = 3, M = 'PT20481' (56 bits), 1024-bit n	M^3 < n, so C = M^3 and the cube root reveals M	cube root of C = PT20481 (no key used)	yes
e = 3, M of 800 bits	M^3 > n wraps modulo n; cube root fails	cube root attack fails	yes
4-digit token, textbook RSA, e = 65537	attacker encrypts all 10,000 guesses and matches	token 7319 found after 7320 encryptions (124440 modular multiplications)	yes

Test	Prediction	Observed result	Match
Same token twice with OAEP	two different ciphertexts, both decrypt to the token	different = True; decrypt = 7319, 7319	yes
Dictionary attack against OAEP (300 guesses)	no ciphertext ever matches	0 matches	yes
One flipped bit in an OAEP ciphertext (60 trials)	decoding error almost always	60 of 60 rejected	yes

What the failed prediction taught. With $p = 91 = 7 \cdot 13$ treated as a prime, I predicted wrong decryption, but all 4,823 messages decrypted correctly: n is square-free and $4680 = 90 \cdot 52$ is accidentally a multiple of $\text{lcm}(6, 12, 52) = 156$. Correct decryption therefore does not show that the key is well made, and here n has factors 7 and 13, so it is trivially weak. The $p = q = 61$ test, where $\varphi = (p-1)(q-1)$ is wrong, decrypted only 61 of 3,721 messages. Together with the deterministic results (same message gives the same ciphertext, and 0, 1, $n-1$ map to themselves) this shows that textbook RSA is structurally weak, not just short.

Factoring gives φ(n) and d. If $n = pq$ is factored, $\varphi(n) = (p-1)(q-1)$ follows by one multiplication, and $d = e^{-1} \bmod \varphi(n)$ by extended Euclid; the private key and every past ciphertext are then open. I factored three toy moduli:

Modulus	Bits	Trial division	Steps	Fermat	Steps
small factor (3 x 61-bit prime)	63	found	2	not found in 1,000,000 steps	1,000,000
close primes (48-bit, gap 48)	96	not found in 3,000,000 steps	3,000,000	found	1
balanced 40-bit primes	80	not found in 3,000,000 steps	3,000,000	not found in 1,000,000 steps	1,000,000

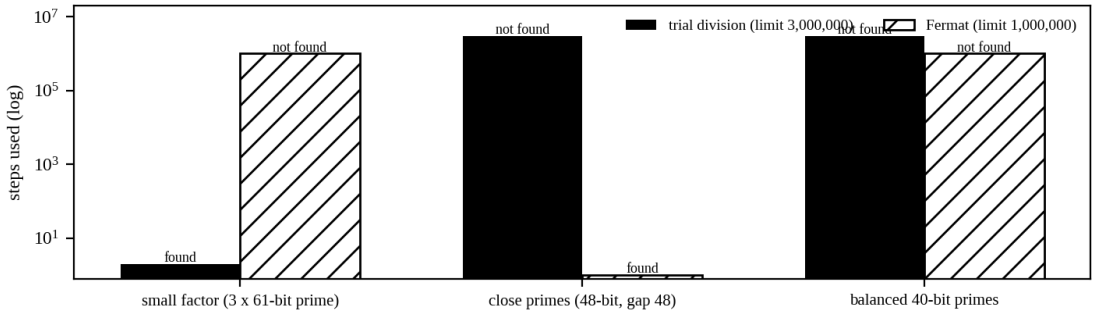


Figure 5. Steps used by trial division and Fermat factoring on three weak toy moduli (log scale).

Attack traces. *Small factor:* trial division finds 3 after 2 steps although n has 63 bits. *Close primes:* $p = 276021347137871$ and $q = 276021347137919$ (gap 48): Fermat's method finds the factors in **1 step**, then $\varphi = 76187784075817784265450754660$, $d = 33423485664034623425179282653$, and a test ciphertext was decrypted correctly (true) without the real key. *Balanced 40-bit primes:* neither method finishes within the limits, which is the expected behaviour for well-spread primes at this size. *Shared prime:* two 1,024-bit moduli built with a repeated prime are separated by one gcd computation of only 304 Euclid steps (common prime recovered).

Key length alone does not guarantee security. The 1,024-bit shared-prime pair and the 96-bit close-prime modulus fall instantly because of how the primes were generated (poor randomness, small or close factors), not because of their size, so key generation must use a secure random generator and the checks of Q7. **Limitations of the experiments:** classroom moduli have at most 96 bits; trial division (about $\sqrt{n}$ steps) and Fermat (cost depends on how close the primes are) are not what attackers use against real keys, where the sub-exponential number-field sieve is the best known method and 2048-bit moduli remain out of reach; step counts at these sizes cannot be extrapolated to real security levels, and only structural weaknesses (small, close or shared factors, small exponent with small messages) are shown.

Padding and OAEP. Textbook RSA is deterministic, multiplicative ($E(M_1) \cdot E(M_2) = E(M_1 M_2)$) and leaks small messages ($e = 3$, M = "PT20481": the cube root of C reveals M with no key; a 4-digit token was found after 7,320 encryptions, 124,440 multiplications). **OAEP** builds the block $EM = 0x00 \,\|\, \text{maskedSeed} \,\|\, \text{maskedDB}$, where $DB = \text{Hash(label)} \,\|\, 00\ldots00 \,\|\, 01 \,\|\, M$ is masked by MGF1(seed) and the random seed is masked by MGF1(maskedDB). The random seed makes encryption **randomised** (the same token gave different ciphertexts, and 300 guesses gave 0 matches), and the two masks make the transform all-or-nothing: 60 of 60 single-bit changes were rejected. Decoding errors must be reported uniformly to avoid padding-oracle attacks.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 marks)

Derivation. Let p be a prime and g a generator (or an element of large prime order q). Alice picks a secret a and publishes $A = g^a \bmod p$; Bob picks b and publishes $B = g^b \bmod p$. Alice computes $B^a = g^{ba}$, Bob computes $A^b = g^{ab}$; since $(g^a)^b \equiv (g^b)^a \pmod p$, both obtain $K = g^{ab} \bmod p$, while an observer who knows p, g, A, B must solve a discrete logarithm. Pseudocode: (1) public: p, g; (2) Alice: a ← random in $[2, q-1]$, A ← POW(g, a, p), send A; (3) Bob: b ← random, B ← POW(g, b, p), send B; (4) Alice: K ← POW(B, a, p); Bob: K ← POW(A, b, p); (5) both: session key ← KDF(K).

Tests. *Hand example:* p = 23, g = 5, a = 6, b = 15: A = 5^6 mod 23 = 8, B = 5^{15} mod 23 = 19; Alice: 19^6 mod 23 = 2; Bob: 8^{15} mod 23 = 2. *Larger toy:* a 64-bit safe prime p = 16183038054000441203 (p = 2q + 1), g = 4 of order q; both sides computed the same key (true), each exponentiation taking 86 multiplications.

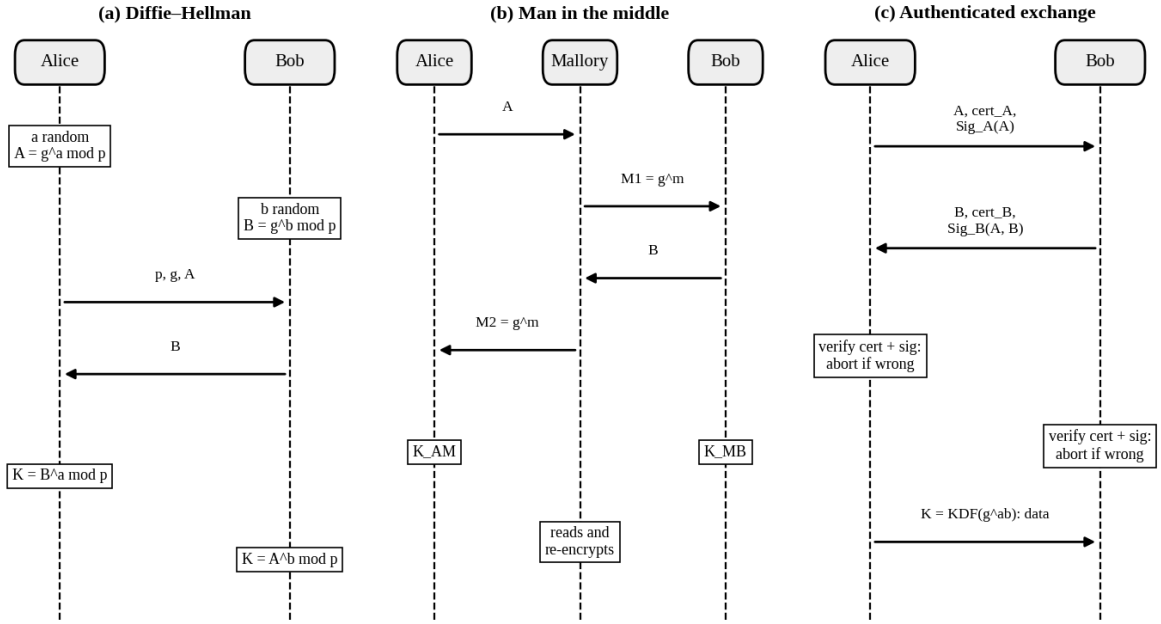


Figure 6. Message-sequence diagrams: plain Diffie–Hellman (a), active man-in-the-middle (b) and authenticated exchange (c).

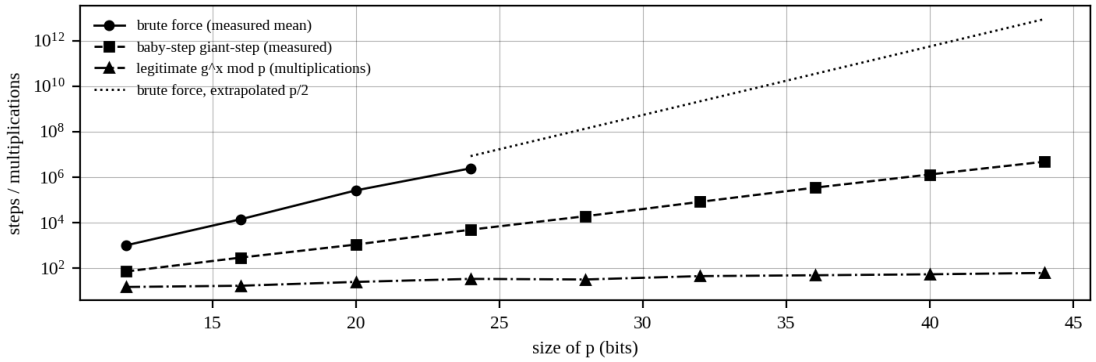


Figure 7. Cost of the legitimate computation and of two discrete-logarithm attacks as p grows.

Bits of p	Brute force (mean steps)	Baby-step giant-step steps	Legitimate exponentiation (multiplications)	Brute force ÷ legitimate
12	1,017	72	15	68
16	14,263	294	17	839
20	261,752	1,092	25	10,470
24	2,438,083	4,927	34	71,708
28	not run (≈ 2^{27})	19,249	32	–
32	not run (≈ 2^{31})	82,367	45	–
36	not run (≈ 2^{35})	345,551	49	–
40	not run (≈ 2^{39})	1,308,678	54	–
44	not run (≈ 2^{43})	4,777,390	62	–

Growth analysis. The legitimate cost grows only like 1.5·$\log_2$ p multiplications (15 → 62 from 12 to 44 bits), brute force doubles with every extra bit (1,017 → 2.4 million steps from 12 to 24 bits) and baby-step giant-step grows by about √2 per bit (72 → 4.8 million steps from 12 to 44 bits). At 2048 bits brute force would need about 2^{2047} steps and baby-step giant-step about 2^{1024}; the best real attack (number-field sieve) is about 2^{112}, which is why p ≥ 2048 bits is required.

Man-in-the-middle demonstration. Mallory replaces A and B by g^m. Alice and Bob each finish happily, but their keys differ (Alice a8683e00…, Bob 6cd86708…) while Mallory holds both (K_AM and K_MB). Alice's order "PT20481 AMOX 500MG TID" was read

by Mallory and delivered to Bob as "PT20481 AMOX 900MG TID", a dose change neither party can notice. Success: **20 of 20** unauthenticated runs. *Defence:* each side signs its public value (and the other's) with its long-term RSA key, whose public half is certified by a CA. With RSA-1024 signatures over the DH values, genuine values verify (true), Mallory's substituted value is rejected (accepted = false), a signature forged with another key is rejected (accepted = false), and the attack succeeded **0 of 20** times. Using fresh (ephemeral) values gave 100 different keys in 100 sessions, so leaking a long-term key later does not expose past sessions (forward secrecy).

Stage	Key-distribution and management policy
Generation	Random numbers from the operating-system generator only; RSA ≥ 3072 bits or P-256/X25519; every modulus passes the weak-key tests of Q7; generated on the device or HSM that will hold it.
Storage	Long-term private keys in an HSM (hospital) or secure element (clinic device), never in plain files; session keys only in memory and wiped after the transfer; backups split between two custodians.
Distribution	Public keys only as certificates from the network CA; enrolment of a new clinic is verified out of band; Diffie–Hellman values always signed.
Rotation	Long-term keys every 12 months (certificates for at most 2 years); session key per file or per session, at most 1 hour and well under 1 GiB per key for 64-bit-block ciphers.
Revocation	Certificate revocation list or online status check; a lost device is revoked within one hour; the hospital refuses handshakes from revoked or expired certificates.
Compromise recovery	Revoke, re-enrol with a new key pair, re-encrypt stored records under new keys, review the audit log for the exposure window; ephemeral session keys limit past damage.
Session-key lifetime	One session or file; never reused; nonce or counter state kept per key so that no counter value repeats.

Q5 – ECC and Public-Key Strategy Comparison (5 marks)

Mathematical basis. An elliptic curve $y^2 = x^3 + ax + b$ over a prime field F_p has points that form a group under the chord-and-tangent rule: for $P \neq Q$, $\lambda = (y_2 - y_1)/(x_2 - x_1)$; for doubling, $\lambda = (3x_1^2 + a)/(2y_1)$; then $x_3 = \lambda^2 - x_1 - x_2$ and $y_3 = \lambda(x_1 - x_3) - y_1$. Scalar multiplication kG is computed by double-and-add, but recovering k from Q = kG (the elliptic-curve discrete logarithm problem) has no known sub-exponential algorithm for good curves, only about √n-step methods.

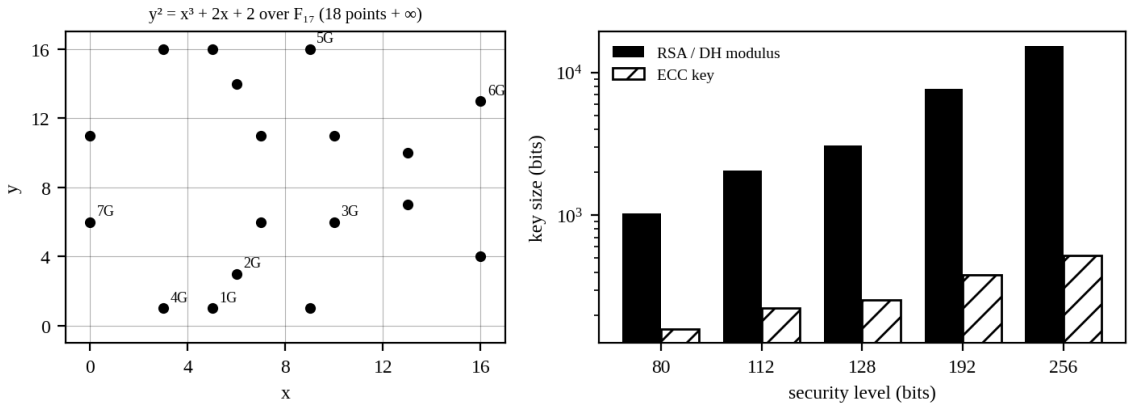


Figure 8. Toy curve over F_{17} with multiples of G = (5, 1) (left); RSA/DH modulus against ECC key size at equal security (right).

Toy example. Curve $y^2 = x^3 + 2x + 2$ over F_{17} has 18 affine points plus the point at infinity, and G = (5, 1) has order 19. Doubling: $\lambda = (3\cdot 25 + 2)/2 = 77\cdot 9 \equiv 13$, $x_3 = 169 - 10 \equiv 6$, $y_3 = 13(5 - 6) - 1 \equiv 3$, so 2G = (6, 3) (program: (6, 3)). *Key agreement:* Alice a = 3 gives A = 3G = (10,6); Bob b = 5 gives B = 5G = (9,16); Alice computes 3·B = (3,16), Bob 5·A = (3,16), both equal 15G = (3, 16). On secp256k1 the generator lies on the curve (true), n·G is the point at infinity (true), and ECDH agrees (true); one scalar multiplication takes 384 point operations.

Property	RSA	Diffie–Hellman (finite field)	ECC (ECDH / ECDSA)
Hard problem	factoring n	discrete log in F_p	elliptic-curve discrete log
Key size for 128-bit security	3072-bit modulus	3072-bit p	256-bit curve
Public value / signature	384 bytes / 384 bytes	384-byte public value	32–64 bytes / 64 bytes
Key generation (measured)	two primes: 0.87 s (2048), 0.98 s (3072)	safe prime found once; a random exponent per session	random scalar and one multiplication: 19.5 ms
Private operation (measured, own Python)	5.2 / 22.9 / 84.8 ms at 1024 / 2048 / 3072 bits	about the same as RSA private operation	19.5 ms for 256 bits (unoptimised affine code; optimised code is much faster)
Public operation	0.1 ms (e = 65537, 17 multiplications)	one exponentiation per side	one scalar multiplication (about as costly as the signing operation of the same scheme)

Property	RSA	Diffie–Hellman (finite field)	ECC (ECDH / ECDSA)
Deployment	universal, simple verification, large keys	needs safe primes and authentication	small keys, fast; needs curve validation; standard curves only

Why ECC needs smaller keys. For RSA and finite-field DH the number-field sieve runs in sub-exponential time, so the modulus must grow fast (3072 bits for 128-bit security). For a well-chosen elliptic curve the best generic attack needs about $\sqrt{n}$ group operations, so a 256-bit group already gives about 2^{128} work (Fig. 8, right), and 32–64-byte keys and signatures reduce bandwidth and storage on constrained devices.

Implementation requirements. (1) *Secure randomness:* ECDSA signing with a repeated nonce k leaked the private key in my test (two signatures with the same k, key recovered = true). (2) *Parameter validation:* check that a received point lies on the curve and has the right order, otherwise invalid-curve and small-subgroup attacks apply. (3) *Protected private keys* in a secure element, constant-time scalar multiplication, standard curves (P-256, Curve25519), and never a home-made curve.

Recommendation for constrained devices. Use ephemeral ECDH (X25519 or P-256) for key agreement and ECDSA or Ed25519 for signatures, with certificates from the network CA. Evidence: prime search makes RSA key generation 0.1–1 s here against one scalar multiplication for ECC (Q2); modular exponentiation cost rises steeply with modulus size (5.2 → 22.9 → 84.8 ms, Q2), while ECC keeps 32-byte keys; discrete-log attacks grow exponentially with the field size (Q4), which ECC exploits best; and unauthenticated exchange is broken by MITM 20 of 20 times (Q4), so every exchange must be signed. RSA-3072 stays at the hospital for compatibility, because its public operation is very cheap for verification.

Q6 – Digital Signatures, Hybrid Encryption and Quantum Readiness (7 marks)

Hash-then-sign. Signing: H = SHA-256(M); EM = 0x00 01 FF…FF 00 ‖ DigestInfo ‖ H; $S = EM^d \bmod n$. Verification: $EM' = S^e \bmod n$ is compared with the EM rebuilt from the received message, and any changed message bit changes H. Signing the fixed-size digest makes the cost independent of message length and protects the whole message.

	RSA signature	DSA / DSS	ECDSA
Keys	n = pq, public e, private d	public parameters (p, q, g); private x, public $y = g^x \bmod p$	curve, G of order n; private d, public Q = dG
Signing	$S = EM^d \bmod n$ (deterministic in PKCS#1 v1.5)	random k; $r = (g^k \bmod p) \bmod q$; $s = k^{-1}(H + xr) \bmod q$	random k; $r = x(kG) \bmod n$; $s = k^{-1}(H + rd) \bmod n$
Verification	one cheap exponentiation (e small)	two exponentiations	two scalar multiplications
Randomness	none needed (PSS adds a salt)	a fresh secret k per signature; reuse reveals x	a fresh secret k; reuse reveals d (demonstrated in Q5)
Security assumption	factoring / RSA problem	discrete log in a prime-order subgroup	elliptic-curve discrete log

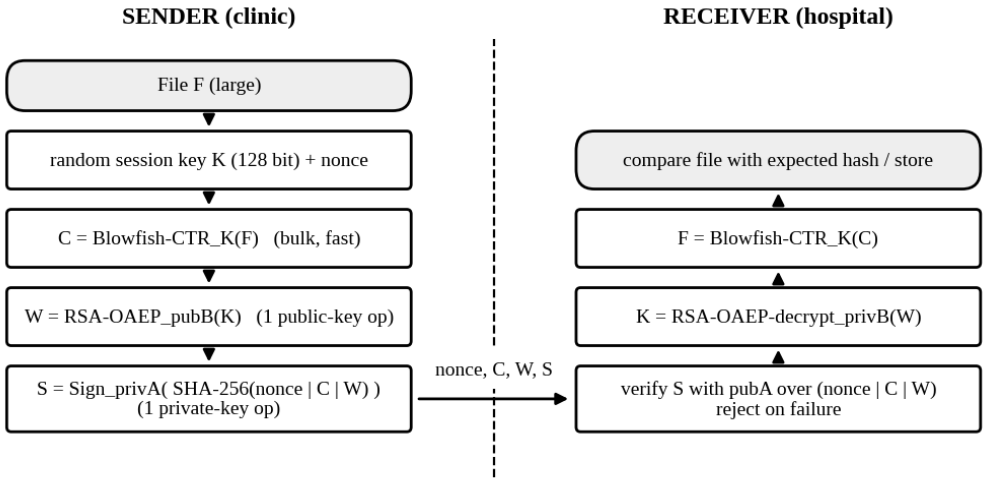


Figure 9. Hybrid encryption with hash-then-sign: only two public-key operations per file for each side.

Hybrid design. The sender draws a random 128-bit session key K and a nonce, encrypts the file with Blowfish in CTR mode (lab cipher; AES-GCM in production), wraps K with the receiver's RSA-OAEP public key, and signs SHA-256(nonce ‖ C ‖ wrapped key) with its own RSA key. The receiver first verifies the signature, then unwraps K and decrypts. The same wrapping step can be replaced by authenticated Diffie–Hellman (Q4), which also gives forward secrecy.

Tamper test (2048-bit keys, 4 KB file)	Result
No tampering	accepted, file identical
One bit flipped in the encrypted file	rejected: signature check failed
One bit flipped in the signature	rejected: signature check failed
One bit flipped in the wrapped session key	rejected: signature check failed
Session key replaced by attacker's own key	rejected: signature check failed
Encryption only (no signature): flip ciphertext to change dose	decrypts silently to 'AMOX 900MG'

Every modification of the file, the signature or the wrapped key was rejected because the signature covers all three parts. The last row shows why the signature is essential: encryption alone is malleable, and a flipped ciphertext bit silently changed “500MG” into “900MG”.

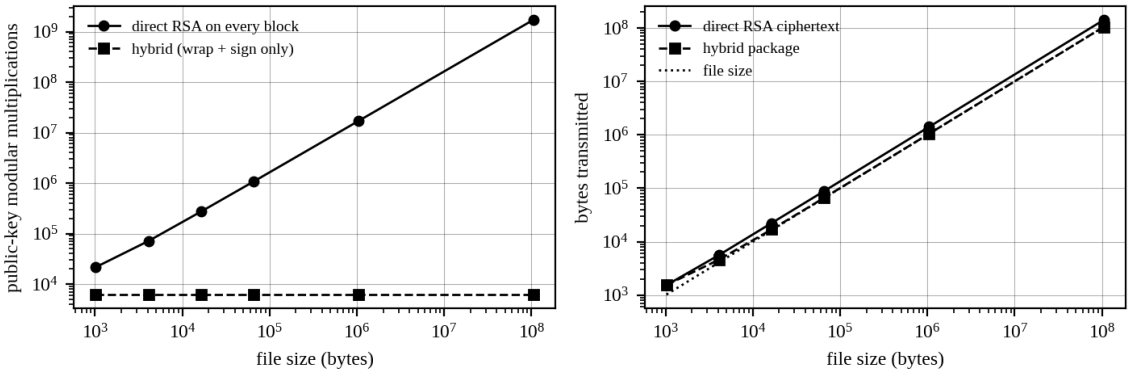


Figure 10. Public-key multiplications (left) and bytes transmitted (right) for direct RSA encryption and hybrid encryption.

File size	RSA blocks (190 B each)	Direct: public-key multiplications	Hybrid: public-key multiplications	Hybrid symmetric round operations	Direct bytes sent	Hybrid bytes sent
1 KiB	6	21,721	6,206	2,048	1,536	1,540
4 KiB	22	71,369	6,206	8,192	5,632	4,612
16 KiB	87	273,064	6,206	32,768	22,272	16,900
64 KiB	345	1,073,638	6,206	131,072	88,320	66,052
1 MiB	5,519	17,128,560	6,206	2,097,152	1,412,864	1,049,092
100 MiB	551,883	1,712,496,052	6,206	209,715,200	141,282,048	104,858,116

Growth comparison. Direct RSA-2048 with OAEP carries only 190 bytes per operation, so the number of operations, the time and the ciphertext grow linearly with file size: 1,073,638 multiplications for 64 KiB, about 1.712×10^9 for 100 MiB (551,883 private operations, roughly 3.4 hours at 22.9 ms each) and the ciphertext is 1.35 times the file. Hybrid encryption uses a constant 6,206 multiplications (two public and two private operations) and 516 bytes of overhead. Measured times with my code: 1 KiB 215 ms direct against 89 ms hybrid; 4 KiB 794 ms against 88 ms; hybrid 16 KiB 120 ms and 64 KiB 240 ms, so beyond about 1 KiB the hybrid time is dominated by the fixed public-key cost.

Quantum key distribution (BB84). The sender encodes random bits on single photons in randomly chosen bases (rectilinear or diagonal); the receiver measures in random bases; they publicly compare bases and keep only the positions where they agree (about half). An eavesdropper must measure without knowing the basis and cannot copy the state (no-cloning), so she guesses wrongly half of the time and disturbs the photon, creating about 25% errors in the kept bits. In my simulation of 4000 photons: without an eavesdropper 1991 bits were kept with 0.0% errors; with an intercept-resend eavesdropper 2036 were kept with **23.6%** errors (480). Comparing m random key bits detects her with probability $1-(3/4)^m = 94.4\%$, 99.7% and 99.999% for m = 10, 20, 40.

Quantum threat and readiness. A large fault-tolerant quantum computer running Shor’s algorithm factors integers and computes discrete logarithms in polynomial time, which breaks RSA, finite-field DH, ECDH and ECDSA completely at any practical key size; Grover’s algorithm only halves the effective bits of symmetric keys, so AES-256 and SHA-256 remain acceptable, while 64-bit blocks and 128-bit Blowfish keys do not. Records with a retention of decades are exposed to “harvest now, decrypt later”. NIST’s transition plan (IR 8547) deprecates quantum-vulnerable public-key algorithms after 2030 and disallows them after 2035, and its standards are ML-KEM (FIPS 203) for key establishment, ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) for signatures. **Strategy:** (1) build an inventory of every RSA/ECC use; (2) make algorithms replaceable by configuration (crypto-agility); (3) move to AES-256 and SHA-256 or better; (4) use hybrid key establishment (X25519 + ML-KEM-768) first, because confidentiality is the long-lived risk; (5) add hybrid certificates and ML-DSA signatures; (6) retire RSA/ECC by 2030–2035 and test the larger objects on constrained devices (ML-KEM-768 public key 1,184 bytes and ciphertext 1,088 bytes; ML-DSA-65 signature about 3.3 KB). QKD does not replace this: it needs dedicated fibre or trusted

relays, it cannot authenticate (it relies on an authenticated classical channel), it gives no signatures, and it does not remove key storage, rotation and revocation, so ordinary key management remains.

Option	Performance and key-management burden	Security	Long-term risk
A. Direct RSA on every block	linear in file size (1.7×10^9 multiplications for 100 MiB); private key used 551,883 times per file	no integrity without extra signature	quantum-broken; costs rise with key size
B. Hybrid RSA-OAEP wrap + RSA signature	constant 6,188 multiplications measured; one session key per file	tamper tests all rejected; no forward secrecy	RSA quantum-broken; migrate
C. Signed ephemeral ECDH + AEAD (PQ-hybrid ready)	two scalar multiplications per side; small keys; PKI needed	MITM 0 of 20, forward secrecy, integrity from AEAD	switch to ML-KEM/ML-DSA by configuration

Deployment decision. Option C is the target architecture (Fig. 11), with option B retained only for legacy sites during migration, because C gives the lowest bulk cost, small keys for constrained devices, forward secrecy and the easiest path to post-quantum algorithms.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 marks)

A. Diagnose the Bottleneck or Weakness (2 marks)

Limitation: public-key work grows with data volume. In the original design every data block was encrypted directly with RSA (and every exchange used unauthenticated DH). The responsible operation is the modular exponentiation performed once per 190-byte block: $\lceil N/190 \rceil \times (17 + 3{,}086)$ multiplications for RSA-2048, plus one signature and its verification (3,103), i.e. **21,721** multiplications for 1 KiB, **1,073,638** for 64 KiB and **1,712,496,052** for 100 MiB (Q6), linear in N. Its cost per operation also grows steeply with key size (5.2 → 22.9 → 84.8 ms for 1024 → 2048 → 3072 bits), so the move to 3072 bits needed for 2030 and beyond multiplies the problem. The missing security property is **authentication of the exchanged values**: unauthenticated Diffie–Hellman was broken by a man in the middle in 20 of 20 runs (Q4). Two further weaknesses seen earlier: ECB leaves 2038 of 2048 blocks as visible repeats (Q1) and a shared prime between two 1,024-bit moduli is exposed by a 304-step gcd (Q3).

B. Define the Design Objective (1 mark)

Measurable objectives: (1) public-key multiplications per file independent of file size and at most 2 public plus 2 private operations (about 6,200 multiplications), a reduction of at least 99% for files of 64 KiB and above; (2) man-in-the-middle success from 20 of 20 to 0 of 20; (3) the weak-key detector rejects 100% of planted weak moduli with 0 false positives; (4) 0 repeated ciphertext blocks; (5) a fresh session key per file and per session, so the long-term private key is used 2 times per file instead of 551,883 times for 100 MiB.

C. Design the Revised Strategy (3 marks)

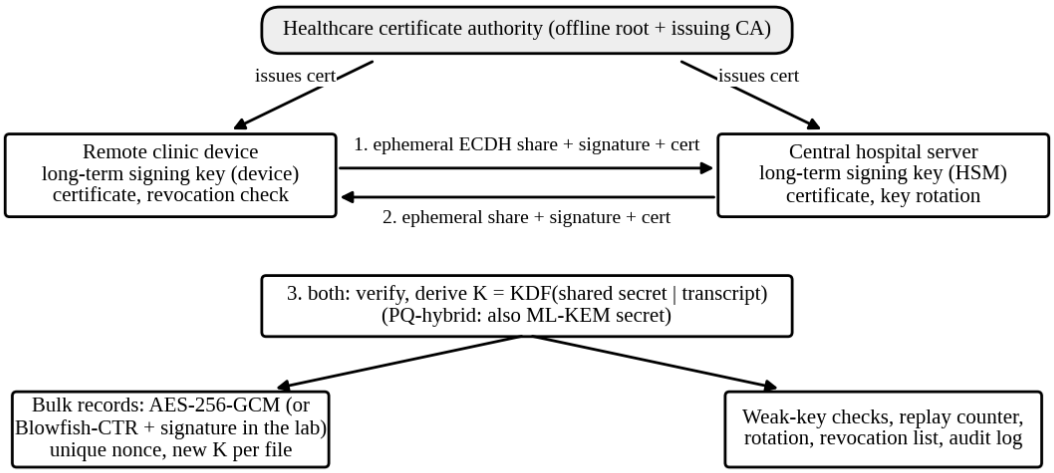


Figure 11. Revised architecture: certificates, signed ephemeral key agreement, derived session key, authenticated bulk encryption and key-management controls.

Component	Property exploited	Operations reduced	New state or keys	New trade-off
Hybrid encryption (random session key per file)	symmetric cost is per byte, public-key cost is per file	public-key work per file from ⌈N/190⌉ operations to a constant 2 + 2	one 16-byte key per file, nonce counter per key	nonce must never repeat (shown dangerous in Q1)
Secure mode (CTR, GCM in production)	unique nonce randomises output	removes repeated-block leakage (2038 → 0)	nonce or counter state	needs integrity: signature or GCM tag
Signed ephemeral ECDH/DH	$(g^a)^b = (g^b)^a$; signature binds the values to an identity	no long-term key use for data; MITM 20 → 0	device and server signing keys, certificates, revocation list	PKI cost, clock and certificate management
Digital signature over the package	hash-then-sign covers file, nonce and wrapped key	all five tamper tests rejected	verification keys at each site	signature size 256 bytes
Weak-key detection at key creation	small factors and close primes are cheap to test; gcd across keys finds shared primes	rejects weak moduli before use	batch store of public moduli	small extra generation time
Key management policy (Q4)	limits exposure in time and scope	rotation, revocation and recovery events defined	HSM / secure elements	operational effort

D. Integrate Quantum Readiness (1 mark)

Threatened dependencies: the RSA-OAEP key wrap, RSA and ECDSA signatures, certificates and (EC)DH key agreement all fall to Shor's algorithm; the symmetric cipher and SHA-256 do not (use AES-256 and 256-bit hashes; Blowfish and 3DES have small blocks and are retired in production). **Migration approach:** phase 0 inventory and crypto-agility (now); phase 1 hybrid key establishment X25519 + ML-KEM-768 for all new sessions (protects against harvest-now-decrypt-later); phase 2 hybrid certificates and ML-DSA signatures; phase 3 remove RSA/ECC by 2030–2035, in line with NIST's transition plan, after testing larger keys and signatures on the constrained clinic devices. **QKD** could add eavesdropping detection on a dedicated fibre between the hospital and a few fixed sites; it **cannot replace** authentication, signatures, certificates, key storage, rotation, or protection of remote clinics over the public network, so it is at most a supplement.

E. Predict, Implement and Validate (2 marks)

Prediction made before implementation: direct cost = ⌈N/190⌉ × (17 + 3,086) multiplications; hybrid cost = 2 × (17 + 3,086) = 6,206 multiplications for any N; the hybrid should therefore need about 11 times fewer operations at 4 KiB, 43 times fewer at 16 KiB and 172 times fewer at 64 KiB (the direct figure excludes the one signature).

File size	Original: predicted multiplications	Original: measured	Redesign: measured (predicted 6,206)	Reduction	Time original / redesign (ms)	Decrypted = original
1 KiB	18,618	18,618 (formula)	6,188	66.8%	215 / 89	yes
4 KiB	68,266	67,870	6,188	90.9%	794 / 88	yes
16 KiB	269,961	268,395	6,188	97.7%	3140 (calc.) / 120	yes
64 KiB	1,070,535	1,070,535 (formula)	6,188	99.4%	12451 (calc.) / 240	yes

Attack or event (Q1–Q4 evidence)	Original design	Redesign	Verification
Man-in-the-middle on key exchange (20 runs)	20 of 20 succeed	0 of 20 succeed	forged value and forged signature both rejected
Repeated ciphertext blocks in the test image	2038 of 2048 (ECB)	0 (CTR with unique nonce)	CBC also 0
Weak RSA moduli (60 good, 60 planted weak)	accepted	60 of 60 caught (20 small factor, 20 close primes, 20 shared prime); 0 false positives	detector: trial division to 2^{20}, Fermat 10^5 steps, pairwise gcd
Tampering with file, signature or wrapped key	undetected without a signature	all rejected	five tests (Q6)
Long-term private-key uses for a 100 MiB file	551,883	2	key-management exposure down by a factor of 275,942
Session keys (100 ephemeral sessions)	one fixed key pair	100 distinct keys	forward secrecy

F. Defend the Final Design

Decision. Adopt signed ephemeral ECDH (or DH) with certificates, a per-file session key and an authenticated mode, with the hybrid RSA package as a transitional option, and make every algorithm replaceable. **Against the alternatives:** direct RSA (A) is correct but scales linearly and needs a private-key operation for every 190 bytes; hybrid RSA (B) fixes the bulk cost (99.4% fewer multiplications at 64 KiB, 100% at 100 MiB) but has no forward secrecy and depends on RSA; the plain DH alternative is broken by a man in the middle every time unless signed. **Residual risks:** a compromised CA or stolen device key (mitigated by HSMs, short certificate lifetime and revocation within an hour); nonce reuse, which I showed to be disastrous (use counters and one key per file); side channels and implementation errors.

Costs and usability: a small PKI, secure elements for devices, a courier-free enrolment process and about one day of training; the cost is lower than the savings in bandwidth and server time for large files. **Weak keys and key compromise:** weak moduli are rejected at creation, and a stolen signing key affects only new authentications, not past session keys. **Long-term risk:** RSA and ECC remain exposed to quantum computers, so the design is deliberately crypto-agile, and the lab ciphers (Blowfish, RC5, DES) must be replaced by AES-256-GCM in any real deployment.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.